CLINIC MISSION CONTROL · RESUME ARTIFACT

Continue here

Max-upgrade audit summary, real measured numbers, open issues and exact next steps

31 October 2026 · repository **Botuserjpg/clinic-enquiry-desk** · branch **main** · audited build **8a51a83**

AUDIT COMPLETE | PUSHED | GITHUB PAGES VERIFIED | OWNER TODOS REMAIN

> **Read this first.** This PDF records what is actually done, the real measured numbers, what remains, and the exact commands needed to resume. It is not a pitch and not a substitute for the safety case. Owner-only work is listed separately and has not been invented.
>
> Live demo: **https://botuserjpg.github.io/clinic-enquiry-desk/**
> Repository: **https://github.com/Botuserjpg/clinic-enquiry-desk**

1. Current state at a glance

Area	Verified state on 31 October 2026
Product	Working single-page clinic enquiry workspace: Inbox, Triage, Queue, Dashboard, Simulator, Rules Lab, Safety Net, Prompt Lab and Settings. Static root, GitHub Pages subpath and `file://` are supported. A local Node server mode is also present.
Data	All messages and clinic details are synthetic. The evaluation set has **1,500** generated messages with labels stored separately. No real patient data, no secrets.
Test status	**21/21** JavaScript/MJS files pass `node --check`; **28/28** Node tests; **27/27** Python unittest; **27** pytest tests plus **12** subtests.
Evaluation	**91.87%** exact category+priority, **95.6%** category, **92.93%** priority; emergency recall **232/232 (100%)**; **50** emergency false positives; **40.8%** human-review rate.
Browser	Static root, static subpath, `file://`, local server and the live GitHub Pages URL all passed the headless Chrome harness. Static modes made **0** API requests; all modes had **0** unexpected external requests, console errors and page exceptions.
Publication	Commits `f79637e`, `4eea86c` and `8a51a83` are on `main`. The Pages workflow run `41165344764` succeeded and the live build was rechecked in Chrome.
Owner work	Prompt 04, a factual prompt row, owner reflections, original prompt history, the personal video, the incognito check and the submission email are still open. See Section 6.

2. What this audit changed

Every item below was reproduced or source-verified before it was changed; each has a regression test or a browser assertion where practical.

- **Server emergency approval was rejecting the app's own template.** The server compared against a shortened hard-coded 000 sentence while the shared rules returned the full configured template with the follow-up sentence. The server now uses `rules.replies.emergency`, and the test rejects the shortened copy.
- **CSV and FHIR exports always said "not approved".** They read a property that never existed on enquiry rows. CSV now joins `reply_approved` events by enquiry ID; the FHIR export receives the joined boolean.

- **Staff names used blocking browser prompts.** Emergency acknowledgement and reply approval now use inline, labelled fields with a 60-character limit, `aria-describedby`, inline alert errors and focus handling. No `window.prompt` remains.
- **"Clear local data" left an approvable result visible and deleted saved rules.** It now resets the result panel, pipeline, tamper marker and history, and keeps the separately saved Rules Lab override.
- **Rules Lab started from authored rules after a reload.** It now starts from the active rules, including a saved override, and resets to that active baseline.
- **Broken and unnamed controls.** The brand link now targets the real `#view-inbox` anchor; reduced-motion and notification switches have explicit accessible names.
- **Docker exposed a port the server did not bind.** `server.js` now accepts `HOST`; the container sets `HOST=0.0.0.0`. Loopback remains the default outside the container.
- **The queue silently cut off at 60 rows.** It now states "Showing the 60 most recent of N matching enquiries" and points to CSV for the full history.
- **Documentation was stale or malformed.** `WHATS_NEW.md` had a BOM, mojibake, a section before the H1 and a "12 things" heading. It is clean UTF-8, starts with the H1 and lists 14 current items. Shipped verification text now names Chrome, matching the harness.
- **Stale evidence was removed.** Four superseded screenshots were deleted; nine current screenshots and four local-mode reports plus the live Pages report remain.

> **Deliberately unchanged:** the working single-page shell and module boundaries; the synthetic-data rule; human approval for every reply; emergency wording precedence; the no-CDN/no-build constraint; owner-only reflections, prompts and video. No external service, model call or sending integration was added.

3. Real numbers

EXACT CATEGORY + PRIORITY

91.87%

1,378 of 1,500 rows; 122 errors. 95% bootstrap CI 90.4%–93.2%.

EMERGENCY RECALL

100%

232 of 232 labelled emergency rows. This is the safety gate.

CATEGORY ACCURACY

95.6%

1,434 of 1,500 rows.

PRIORITY ACCURACY

92.93%

1,394 of 1,500 rows.

EMERGENCY FALSE POSITIVES

50

3.94% of non-emergency rows. Over-triage is accepted; a miss is not.

HUMAN-REVIEW RATE

40.8%

612 of 1,500 rows.

Metric	Value	Source / note
Dataset	**1,500 rows**	Seed `20261030`; labels in a separate file. Enquiries SHA-256 `50da75f7…fd5ca6`; labels `1820bf1e…2c86d`.
Exact category + priority	**91.87%**	`docs/evaluation_metrics.json` and the generated report. 95% CI 90.4%–93.2%, 2,000 bootstrap rounds.
Category / priority	**95.6% / 92.93%**	Per-field accuracy on the same rows. Category and priority are scored separately.
Emergency recall	**232/232**	100% recall on rows labelled Emergency. Negated emergencies are labelled Emergency on purpose.
False positives	**50**	3.94% of non-emergency rows. Accepted over-triage, visible in the report rather than hidden.
Review rate	**40.8%**	Rows routed to human review by the current rules.
Latest latency	**48.27 ms**	0.0322 ms per message; full 1,500-row set in one local Python process. Repeated runs measured 42.98–81.43 ms. Not a service-level claim.

Evaluation is rule-consistency evidence on fictional examples, **not clinical validation**. The classifier over-triages look-alike phrases on purpose. Accuracy is informative; emergency recall is the gate.

4. Verification evidence

Check	Result	What was actually run
JavaScript syntax	**21/21**	`node --check` on every tracked `.js` / `.mjs` file.
Node tests	**28/28**	`npm test` — classifier, rules parity, evaluation, FAQ, slots, duplicate warning, palette search, exports, server permissions, emergency template, service worker, scroll and accessibility regressions.
Python unittest	**27/27**	`python -m unittest discover -s tests -v`.

pytest	**27 + 12**	`python -m pytest -q` — 27 passed, 12 subtests passed. A cache-path warning is environment noise.
Evaluation	**PASS**	`python tools/evaluate.py --report docs/evaluation_report.md`. Regenerated JSON, browser bundle and Markdown report from seed `20261030`.
Static root	**PASS**	Headless Chrome, 0 API requests, 0 external requests, 0 console errors, 0 page exceptions.
Static subpath	**PASS**	Served under `/clinic-enquiry-desk/`; same clean result.
`file://`	**PASS**	Opened directly in Chrome; same clean result. Report URL is redacted to `file:///[redacted]/index.html`.
Local server	**PASS**	Node server on loopback; **25** same-origin API requests from the full harness; 0 external requests, 0 console errors, 0 page exceptions.
Live GitHub Pages	**PASS**	Headless Chrome against the deployed URL: 0 API requests, 0 external requests, 0 console errors, 0 page exceptions; 7 Trust Center checks and the inline identity flow passed.

> **Browser harness scope.** The harness drives real Chrome through the CDP, not a DOM-only simulation. It checks the eight-stage pipeline, FAQ answer, synthetic slots, reply studio, emergency gate, queue, dashboard, Rules Lab full-set gate, seven Trust Center self-tests, simulator, command palette, scroll preservation, reduced-motion handling, inline staff-name validation, emergency acknowledgement, reply unlock and approval. Evidence files are `docs/browser_check_current_static.json`, `..._subpath.json`, `..._file.json`, `..._server.json` and `..._pages.json`.

Some local runs saw requests injected by a desktop antivirus product into localhost pages. The harness reports those separately as environment noise; the application's own unexpected external request count was 0.

5. Open issues and deliberate limits

- **Docker was not built locally.** Docker is not installed in this environment. The `HOST=0.0.0.0` container fix is source-level and covered by the regression test, but the image itself has not been run here.
- **Live Pages verification covers static mode.** Server mode was verified locally on loopback. Pages has no server component by design.
- **No clinical validation.** The rules are transparent text matches. They can miss emergencies and can flag ordinary or negated text. The 50 false positives and 122 exact errors are disclosed in the generated report.
- **No message is sent.** Approval is a local human decision. There is no email, SMS or emergency-service integration.
- **No authentication or production audit storage.** The hash chain can expose some edits; it is not a security boundary.
- **Service worker is HTTPS-only.** On `file://` the app runs without it. Offline fallbacks read only this app's namespaced cache.
- **Owner-only work remains.** The prompt record, reflections, original prompts, personal video, incognito check and submission email were intentionally not fabricated or completed by the agent.
- **Non-blocking CI note.** The Pages workflow passed, but GitHub reports a Node.js 20 deprecation annotation for the action versions used. It did not affect the deploy.

6. Exact next steps

If you are the project owner

1. **Save prompt 04.** Put the actual prompt used for the Max round into the existing `prompt_history/` pattern. Do not reconstruct it from memory if it is unavailable.
2. **Add the factual prompt row.** Record the real provenance in `PROMPTS.md` / the prompt-history index. Do not invent a prompt or a date.
3. **Write your own reflections in `AI_USAGE.md`.** These are owner reflections; the agent has not written them.
4. **Add the original round 01/02 prompts only if you can retrieve them.** `prompt_history/03_wow_prompt.md` is already saved. If the old prompts are lost, leave the gap visible rather than inventing text.
5. **Record the personal 2+ minute video** in your own words, following `VIDEO_SCRIPT.md` without rewriting it. Add the link to `SUBMISSION_EMAIL.md`.
6. **Run the incognito check.** Open **https://botuserjpg.github.io/clinic-enquiry-desk/** in a private window; confirm the Inbox loads, the emergency gate works, and the Trust Center self-tests pass.
7. **Confirm public visibility.** Check that the repository and Pages site are public, and that no real patient data, secrets or personal paths were added.
8. **Send the submission email** to **jobs@gokulinfocare.com** with the repository URL, live demo URL and the video link.

If you are the next agent

1. `git pull origin main` then `git status --short`. The tree should be clean; the PDF and live-Pages evidence are the final audit commit.
2. Run the four gates:

   ```
   npm test
   python -m unittest discover -s tests -v
   python -m pytest -q
   python tools/evaluate.py --report docs/evaluation_report.md
   ```

   Expected: 28/28, 27/27, 27 + 12 subtests, and the same 1,500-row result. Do not hand-edit evaluation numbers; regenerate them.
3. Read `WHATS_NEW.md`, `docs/FINAL_AUDIT.md` and `docs/evaluation_report.md` before changing behaviour.
4. For browser regression, serve the root and a copy under `/clinic-enquiry-desk/`, run `node server.js` on loopback, then run:

   ```
   node tools/browser_check.mjs --url "http://127.0.0.1:8124/" --out docs/screenshots --report docs/browser_check_current_static.json
   node tools/browser_check.mjs --url "http://127.0.0.1:8125/clinic-enquiry-desk/" --report docs/browser_check_current_subpath.json
   node tools/browser_check.mjs --url "file:///<repo>/index.html" --report docs/browser_check_current_file.json
   node tools/browser_check.mjs --url "http://127.0.0.1:3101/" --server --report docs/browser_check_current_server.json
   ```

   Static modes must make 0 API requests; the server run should make same-origin API calls. All modes must have 0 unexpected external requests, console errors and page exceptions.
5. If behaviour changes, update the tests, the relevant docs and the generated evaluation artifacts in the same change. Keep the static root/subpath/`file://` contract and the no-CDN/no-build constraint.
6. Do not touch `VIDEO_SCRIPT.md`, owner reflections, original prompt text or the personal video. Do not invent owner prompts.
7. Commit, push `main`, confirm the Pages workflow succeeds and recheck the live URL. Record the evidence in `docs/`.

7. Git state and key files

Item	State
Branch	`main`, tracking `origin/main`.
HEAD before this audit	`30942ef` — "Document verified Pages deployment".
Audit commits	`f79637e` — fixes and tests; `4eea86c` — evaluation artifacts and browser evidence; `8a51a83` — documentation.
Pushed build	`8a51a83` on `main`; GitHub Pages deployed successfully in run `41165344764`.
After PDF generation	This PDF, the live-Pages report and the final audit evidence update are committed together. Check `git log --oneline -4` for the exact final hash.
Static network contract	Static root, static subpath and `file://` made **0** application API requests and **0** external requests.

File	Why it matters
`app.js`	Main UI controller: triage, result panel, emergency acknowledgement, reply studio, exports, clear-history reset.
`extras.js`	CSV/FHIR exports, FAQ retrieval, slots, duplicate detection, trends, reply helpers.
`server.js`	Loopback server, permissions, acknowledgement/approval API, exact emergency template, `HOST` binding.
`ruleslab.js`	Full 1,500-row gate; active-rule baseline and saved override.
`views.js`	View switching, queue rendering and the 60-row truncation notice.
`tests/upgrade.test.js`	28 Node regressions, including exports, inline identity, server template and browser contract.
`tests/test_upgrade.py`	27 Python documentation/asset regressions and evaluation checks.
`tools/browser_check.mjs`	Real-Chrome four-mode harness, inline identity flow, screenshots and evidence reports.
`docs/evaluation_metrics.json`	Authoritative evaluation numbers and latest latency. The report and browser bundle are generated from it.
`docs/FINAL_AUDIT.md`	Audit table, current browser evidence and the 31 October fix list.
`BUILD_LOG.md`	Dated record of the real defects found and fixed.
`WHATS_NEW.md`	14-item feature guide with exact click paths and current numbers.
`HANDOFF_STATE.md`	Historical audit plus the current verified status.
`continue.pdf`	This resume artifact.

8. Reproduce the verification

> **Environment used for this audit:** Windows desktop, Node **26.5.1**, Python **3.14.6**, headless Chrome **156.0.8078.76**, Git. Python tools use only the standard library plus pytest; the app has no package dependencies and no build step.

1. Clone the repository and open the root in a terminal.
2. Run `node --check` on every tracked JS/MJS file, then the three test suites and the evaluation command in Section 6.

3. Serve the root on one local port and a copy under `/clinic-enquiry-desk/` on another; start the Node server on a third loopback port.
4. Run the browser harness for static root, subpath, `file://` and server mode. Save reports under `docs/` and screenshots under `docs/screenshots/`.
5. Push `main`; confirm the Pages workflow succeeds; run the harness against the live URL with a cache-busting query string.

If a check fails, fix the cause and rerun all gates. Do not delete a failing test to make the suite green. Do not hand-edit generated evaluation numbers; rerun `tools/evaluate.py`.

> **Safety statement.** This is a synthetic demonstration. It does not diagnose, does not give medical advice and must never contain real patient data. If there is immediate danger, call **000**. Emergency wording always wins over every other rule. Every reply still requires human approval. Nothing is sent.
